

Data modeling

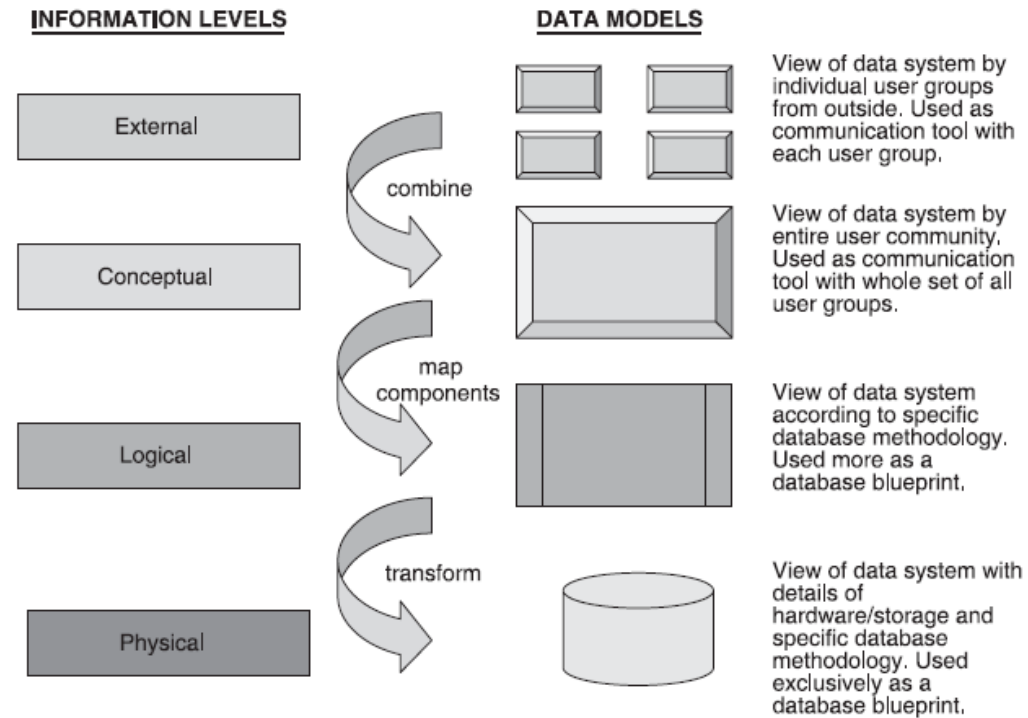
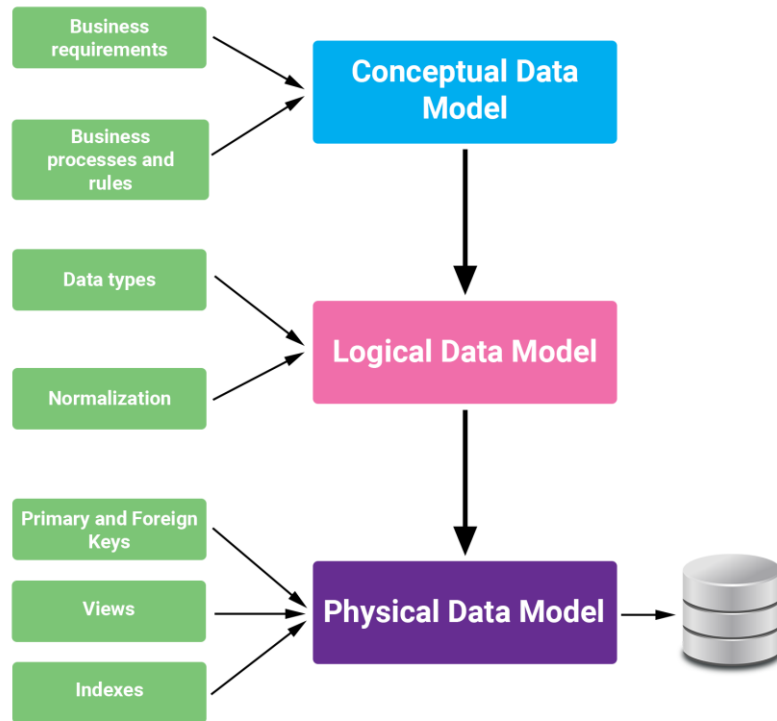
Notation reminder

Data modeling

- Data modeling is the process of creating a visual representation of either a whole information system or parts of it to communicate connections between data points and structures.
- The goal is to illustrate the types of data used and stored within the system, the relationships among these data types, the ways the data can be grouped and organized and its formats and attributes.
- Data models are built around business needs. Rules and requirements are defined upfront through feedback from business stakeholders so they can be incorporated into the design of a new system or adapted in the iteration of an existing one.
 - IBM, <https://www.ibm.com/cloud/learn/data-modeling>

Data modeling

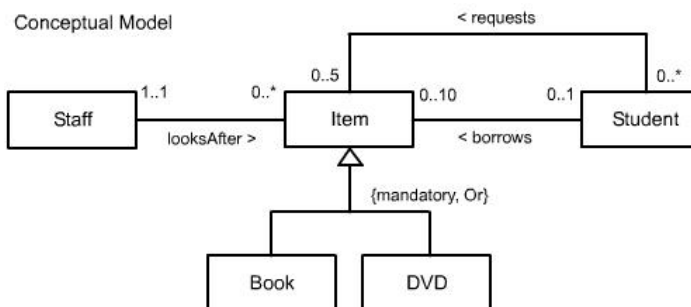
- Data modeling employs standardized schemas and formal techniques. This provides a common, consistent, and predictable way of defining and managing data resources across an organization
- Data models can generally be divided into three categories, which vary according to their degree of abstraction (begins at a high level of abstraction and becomes increasingly more concrete and specific):
 - start with a conceptual model,
 - progress to a logical model
 - and conclude with a physical model.



Data modeling

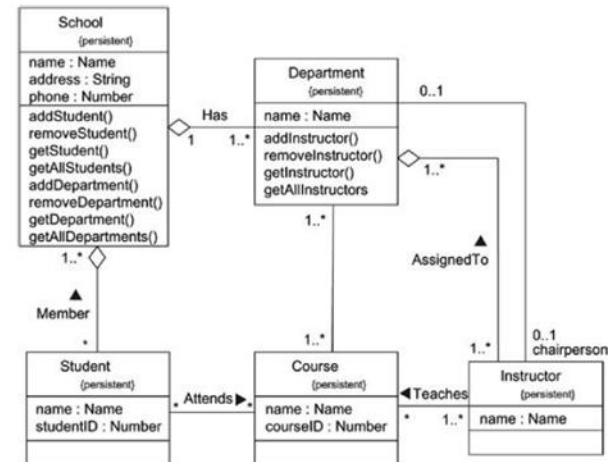
Data modeling

- Conceptual data models.
 - also referred to as domain models
 - offer a big-picture view of what the system will contain, how it will be organized, and which business rules are involved.
 - usually created as part of the process of gathering initial project requirements.
- Typically include entity classes (defining the types of things that are important for the business to represent in the data model), their characteristics and constraints, the relationships between them and relevant security and data integrity requirements.



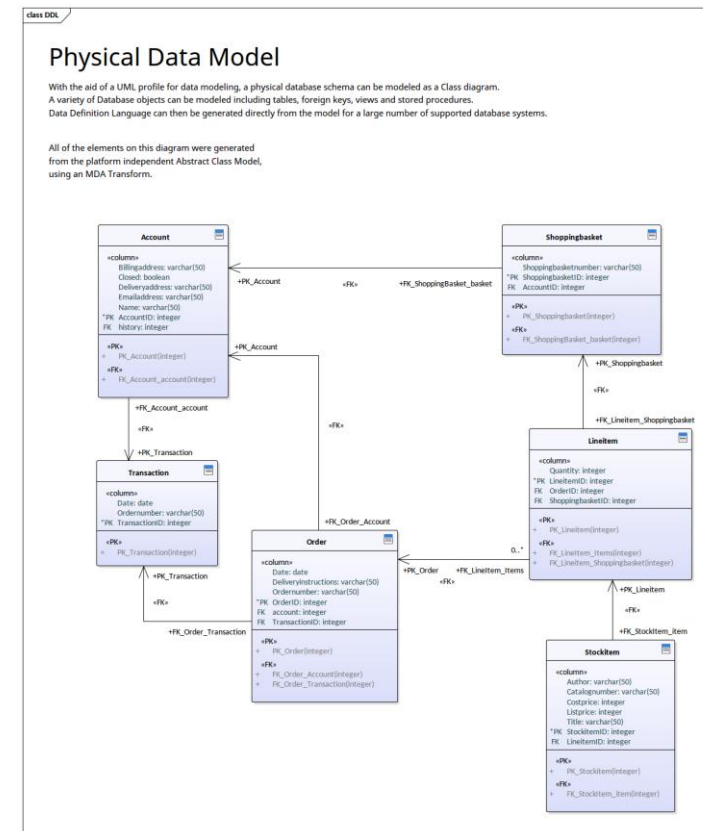
Data modeling

- Logical data models.
 - less abstract and provide greater detail about the concepts and relationships in the domain under consideration.
 - indicate data attributes, such as data types and their corresponding lengths, and show the relationships among entities.
 - don't specify any technical system requirements.
 - can be useful in highly procedural implementation environments, or for projects that are data-oriented by nature.



Data modeling

- Physical data models.
 - provide a schema for how the data will be physically stored within a database.
 - the least abstract of all
 - offer a finalized design that can be implemented as a relational database, including associative tables that illustrate the relationships among entities as well as the primary keys and foreign keys that will be used to maintain those relationships.
 - can include database management system (DBMS)-specific properties, including performance tuning.



Data modeling

- General workflow:
 - Identify the entities.
 - Identify key properties of each entity.
 - Identify relationships among entities.
 - Map attributes to entities completely.
 - Keys and normalization.
 - Finalize and validate the data model.

UML for data modeling

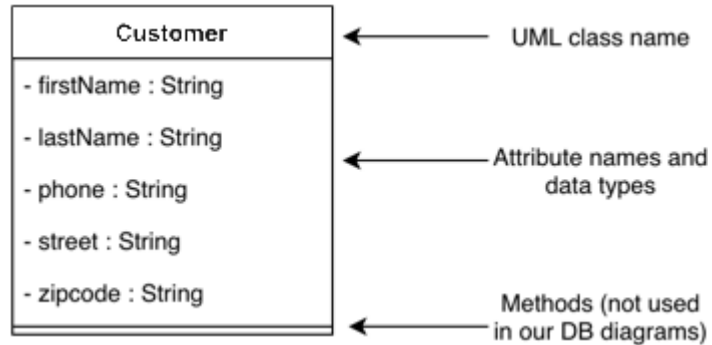
- The way data is organized in object-oriented program is very different from the the way it's organized in relational database
 - Example:
 - OO inheritance is not directly supported in relational database
- OO world needs to be transformed to relational world
 - Solution - use a subset of UML Class diagram's elements

UML for data modeling

- 5 basic concepts:
 - Classes
 - Associations
 - Association classes
 - Subclasses
 - Composition and aggregation

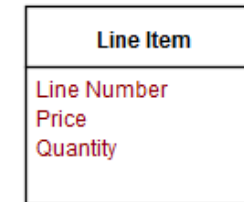
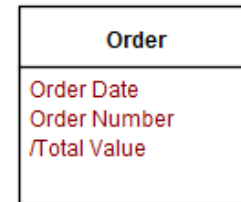
UML for data modeling

- A database has information that is representative of the real world.
 - When we model aspects of the real world, we find that we need to represent a physical “thing” or simply a fact or an event that happens.



UML for data modeling

- Class
 - not specific to data-modeling
 - for data modeling specifically,
 - we add to the attributes the concept of a primary key
 - we drop the methods that are associated.
- attributes can be computed by using values in other attributes
 - in the class diagram we precede their name by a “/” to denote that they are derived attributes.

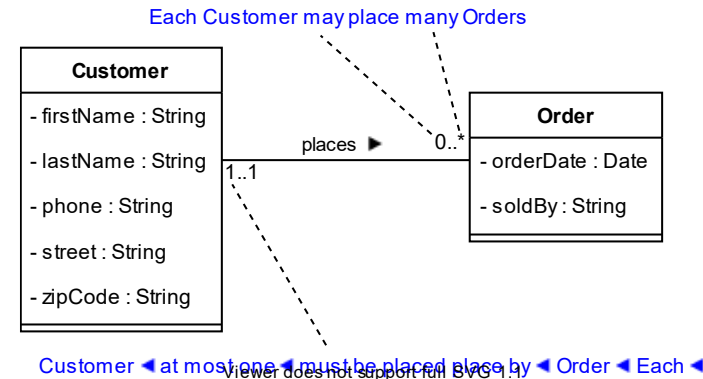


UML for data modeling

- Keys
 - super key
 - is any set of attributes whose values, taken together, uniquely identify each row of a table
 - candidate key
 - is a minimal super key
 - “minimal” means that if you take away any one attribute from the set, it is no longer a super key.
 - primary key
 - can be selected from candidate keys
 - Every table must have at least one candidate key
- There are two types of “made up” primary keys:
 - **surrogate PK** is a single, small attribute (such as a number) that has no descriptive value
 - are created for the convenience of the database designer only.
 - should normally be hidden from the user by the interface of an application that communicates with a database system.
 - **substitute PK** is a single, small attribute that has at least some descriptive value (such as an abbreviation).
 - are also created for the convenience of the database designer.
 - Are less of a nuisance for database users
- *Do not* automatically add “ID numbers” (surrogate keys) or substitute keys to a table until you are *sure* of the following:
 - there is at least one candidate key (before the surrogate is added),
 - the table is a parent in at least one association, and
 - there is no candidate key small enough for its values to be copied many times into the child table.

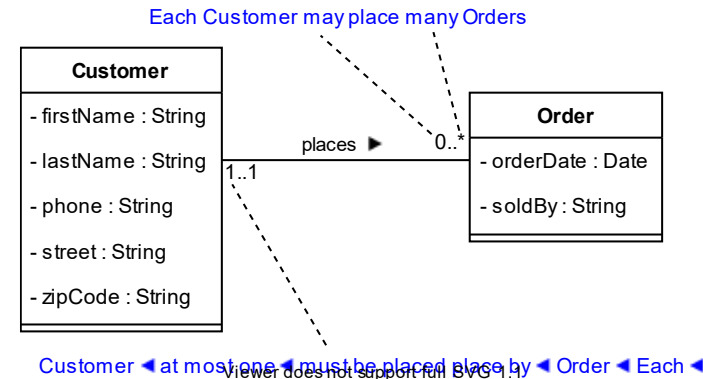
UML for data modeling

- The UML association models the way that two classes are functionally connected to each other
 - it represents the possibility of a relationship between objects of the classes.



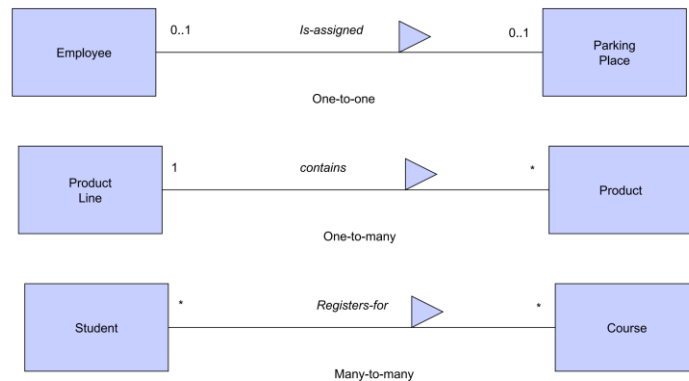
UML for data modeling

- An association
 - can have a name that describes the nature of this relationship.
 - can have a triangle next to this name to indicate the direction in which the name should be read



UML for data modeling

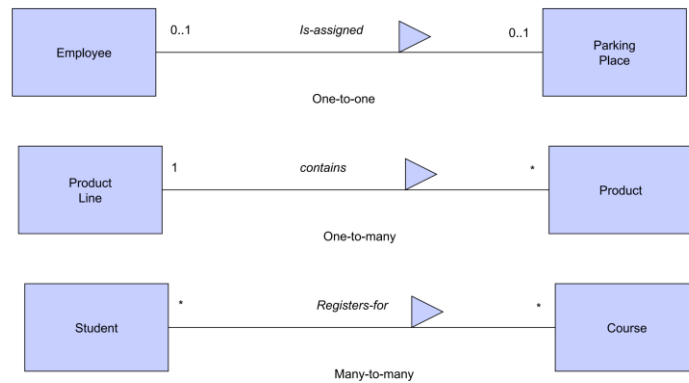
- Describe the association in natural language
 - just as the classes
- Include constraints about how few (at minimum) and how many (at maximum) individuals (objects) of one class may be connected to a single individual of the other class.
- This is called the **multiplicity** of the association
 - Described in both directions.



UML for data modeling

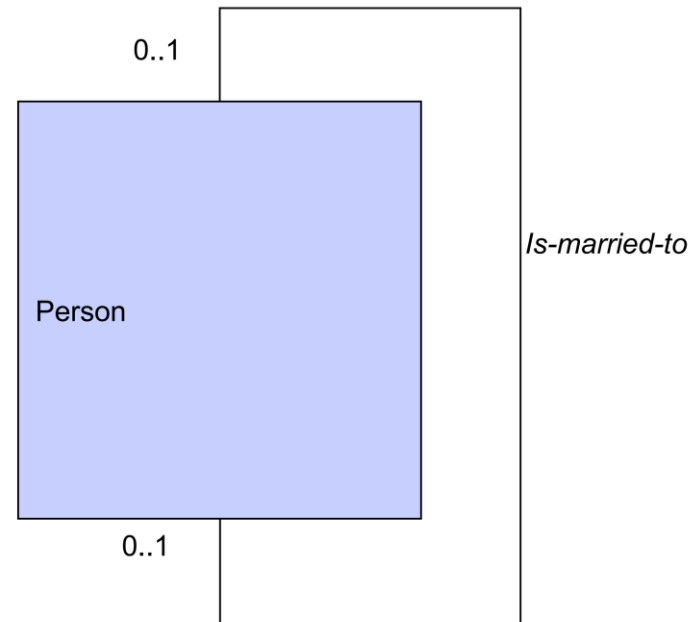
- Multiplicity
 - is a range of the minimum and maximum values
 - syntax: number or min..max

Multiplicity	Notation
zero	0 or 0..0
exactly one	1
zero or one	0..1
zero or more	0..* or *
one or more	1..*



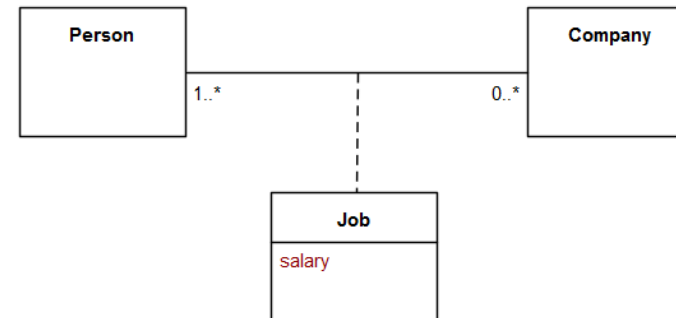
UML for data modeling

- A recursive association connects a single class type (serving in one role) to itself (serving in another role)



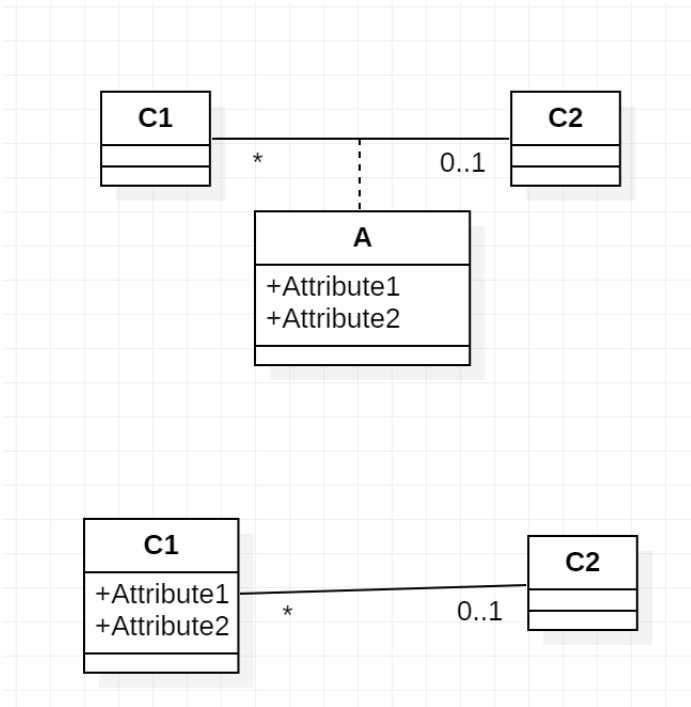
UML for data modeling

- UML gives us a way to model relationship attributes by showing them in an **association class**
- An Association Class
 - is both an Association and a Class
 - assumes at most one relationship between two objects
- Example
 - that tells us that each instance of the association between a person and a company has additional information



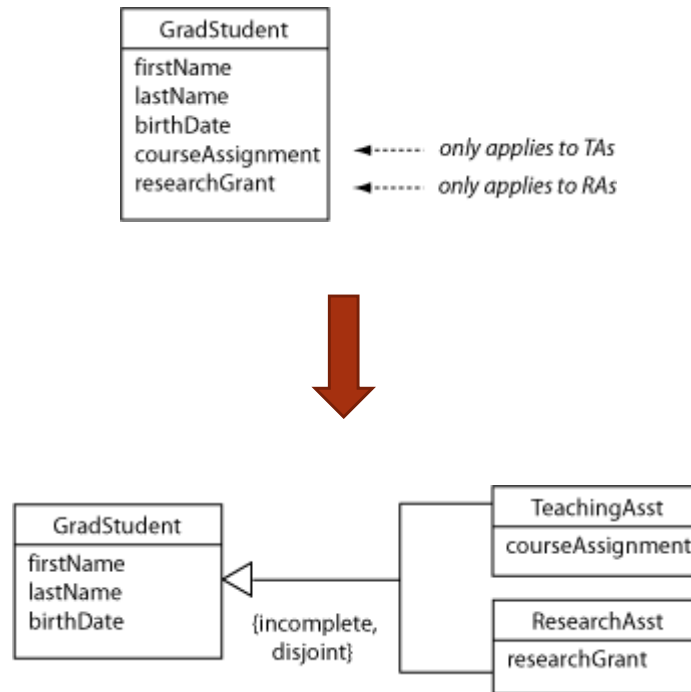
UML for data modeling

- An Association Class
 - unnecessary, if multiplicity is 0..1 or 1..1



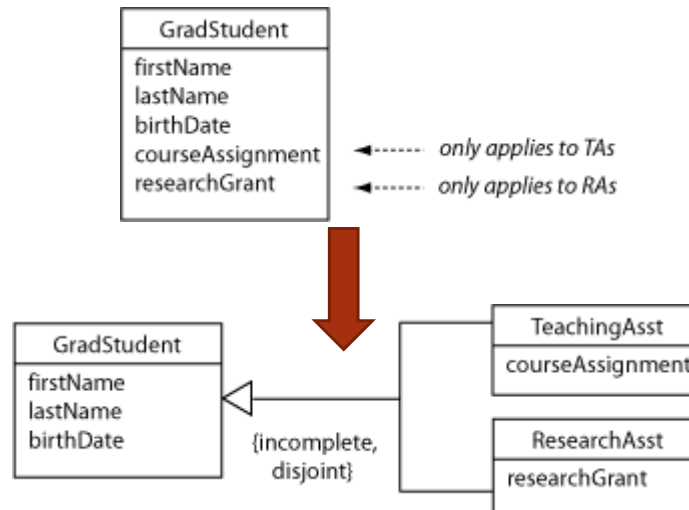
UML for data modeling

- **Subclass** of the basic class type
 - one or more attributes of a class are characteristics of only *some* individuals of that class, but not of others
 - a class that represents a subset of another class type can also be called a specialization of its parent class



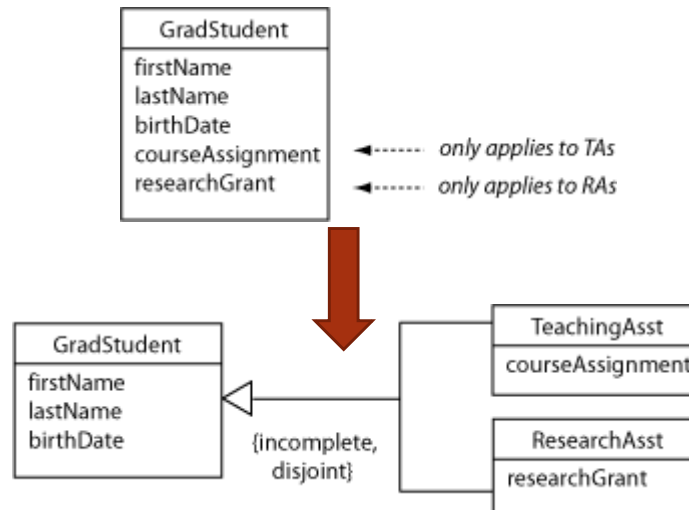
UML for data modeling

- Specialization constraints
 - Rather than the usual cardinality/multiplicity symbols, the subclass association line is labeled with specialization constraints.
 - Constraints are described along two dimensions:
 - incomplete versus complete
 - disjoint versus overlapping.



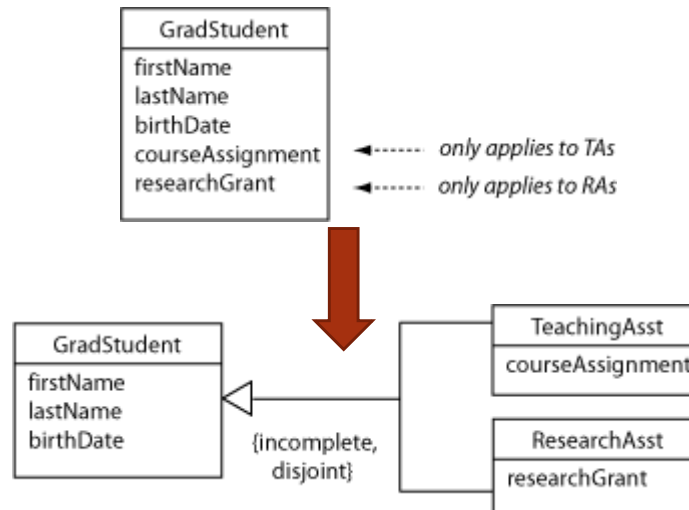
UML for data modeling

- Specialization constraints
 - incomplete specialization
 - also called a partial specialization,
 - only some individuals of the parent class are specialized (that is, have unique attributes)
 - other individuals of the parent class have only the common attributes.
 - complete specialization
 - all individuals of the parent class have one or more unique attributes that are not common to the generalized (parent) class.



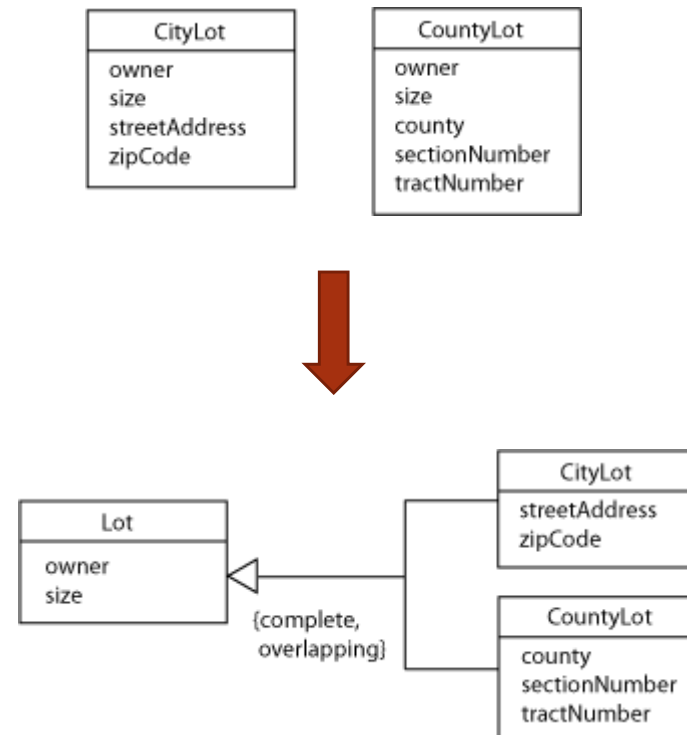
UML for data modeling

- Specialization constraints
 - disjoint specialization
 - also called an exclusive specialization
 - an individual of the parent class may be a member of only one specialized subclass.
 - overlapping specialization
 - an individual of the parent class may be a member of more than one of the specialized subclasses.



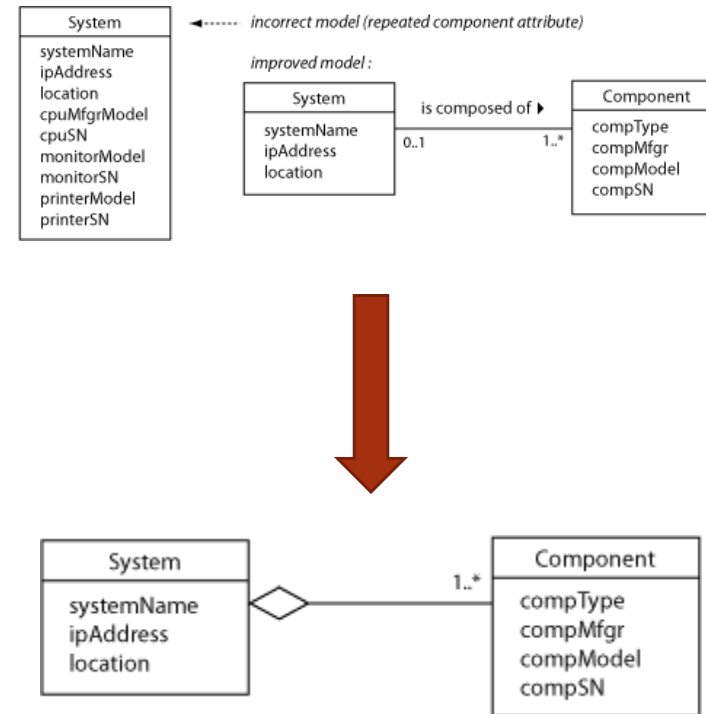
UML for data modeling

- Bottom up design
 - Instead of finding unique attributes in a single class type, you might find two or more classes that have many of the same attributes.
 - Might indicate a need to develop a superclass of the classes with common attributes.
 - The process of designing subclasses from “bottom up” generalization;



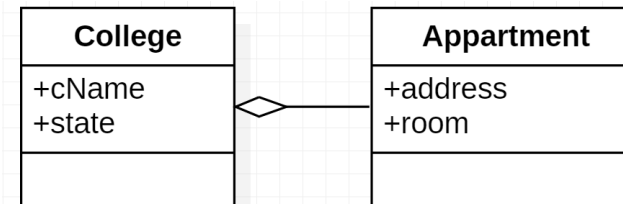
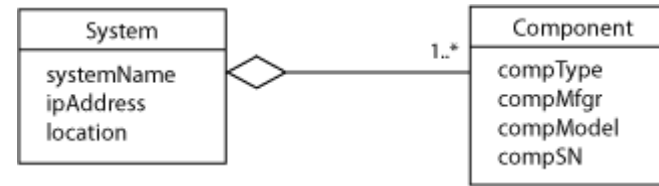
UML for data modeling

- Sometimes a class type really represents a collection of individual components.
- Although this pattern can be modeled by an ordinary association, its meaning becomes much clearer if we use the UML notation for an **aggregation**.



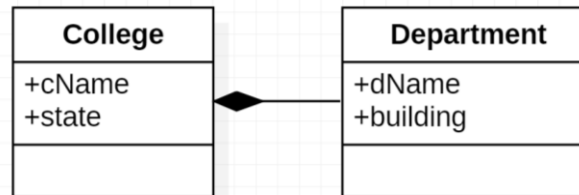
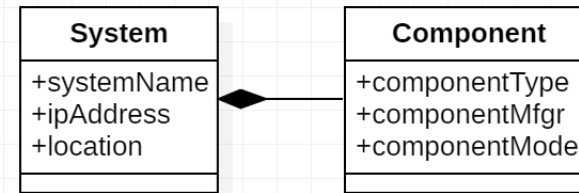
UML for data modeling

- Aggregation is shown by an open diamond on the end of the association line that points to the parent (aggregated) class.
 - There is an implied multiplicity on this end of 0..1, with multiplicity of the other end shown in the diagram as usual.
- Example
 - To describe this association, we would say that each system is composed of one or more components and each component is part of zero or one system.



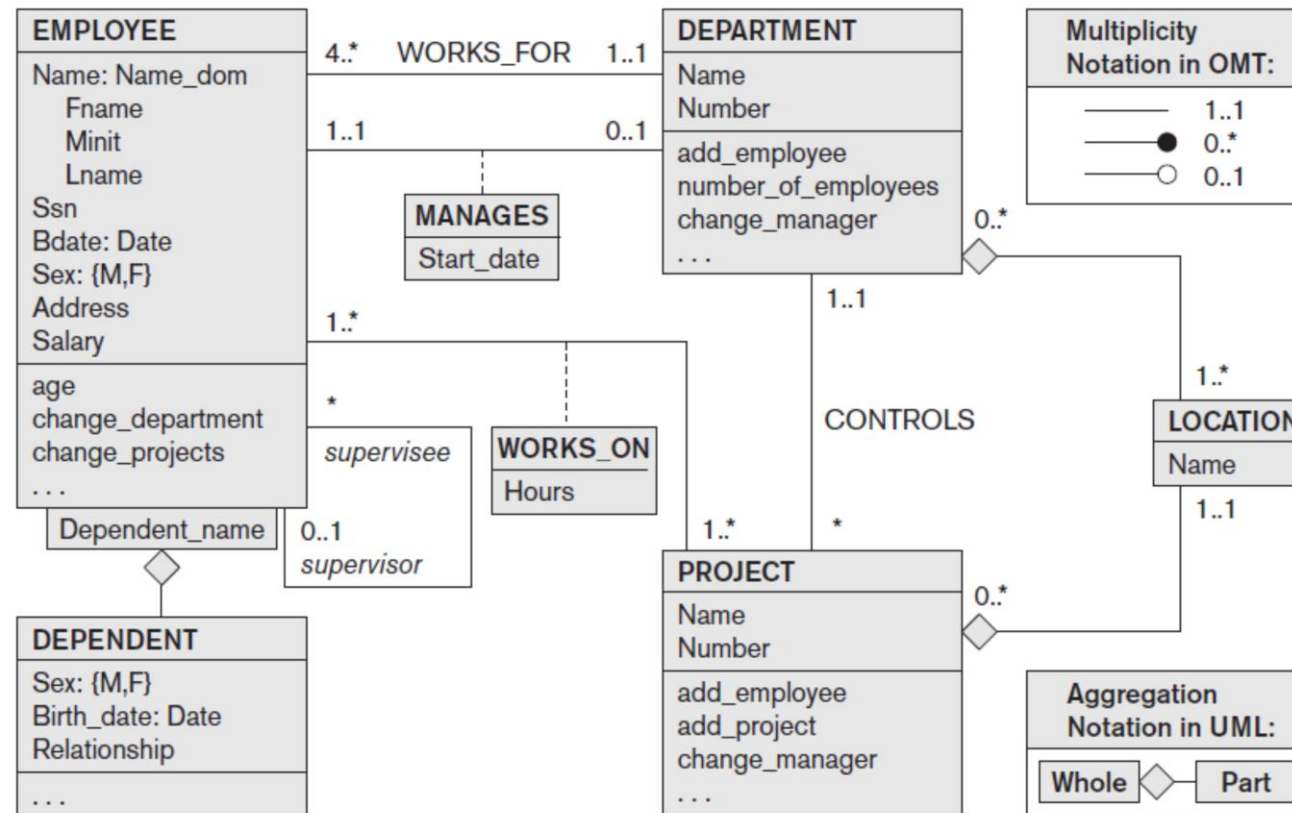
UML for data modeling

- There is a stronger form of aggregation that is called composition.
 - a filled-in diamond
- In composition,
 - component instances cannot exist on their own without a parent
 - they are created with (or after) the parent and they are deleted if the parent is deleted.
- The implied multiplicity on the “diamond” end of the association is therefore 1..1.



UML for data modeling

The COMPANY conceptual schema
in UML class diagram notation.



UML for data modeling

- UML to relations
 - A set of rules and good practices that allows translating UML diagram to a logical/physical database design
 - Require a key for every „regular” class
 - Can be automated
- In short
 - A class becomes a relation
 - An association becomes a relations with key from each side of the association
 - Depends on multiplicity – relation folding
 - Self-associations are still associations
 - An association class has additional attributes added to relation representing association
 - Also depends on multiplicity – relation folding
 - Subclasses – depends on requirements
 - Superclass + subclass with specialized attributes
 - Mostly incomplete
 - Superclass + Subclass contains all attributes
 - Disjoint and complete
 - One mega relation for super and sub classes
 - Heavily overlapping, nulls
 - Composition – „part” class requires a key from „whole” class
 - Analogous to 1..1 association
 - Aggregation – „part” class might have a key from the „whole” class
 - Analogous to 0..1 association, nulls